Information Retrieval

- Information Retrieval (IR) is the field that deals with retrieving relevant information from a large corpus of records given a user search query
- In traditional IR:
  - The records are documents:
  - The search query is a list of words (terms)
  - The output contains the relevant documents for the search query

Information Retrieval and Search Engines
Search Engines

- A **search engine** is an information retrieval system designed to help find information stored on a computer system. (Wikipedia)

Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Keywords queries (search terms – most used): The user uses a list of terms (at list one) with the aim to retrieve documents that contain at least one or all the search terms
    - Soft IR systems – the documents returned contain at least one keyword (logical OR between the terms – added automatically)
    - Hard IR systems – the documents returned contain all the keywords (logical AND between the terms – added automatically)
    - In some IR systems the order of the keywords is also important – returns documents where the list of words appear together
  - Note:
    - Bag of Words (BOW) approach is used when the order is not important
    - N-gram approach is used when the order is important

Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Boolean queries (BOW approach): the user can use Boolean operators in their search queries (AND, OR, and NOT)
    - Example 1: 'data or web' or 'data and web' – different results
    - Example 2: 'data or web and not datamining'

Information Retrieval Architecture
User Queries

- User Queries types:
  - Phrase queries (n-gram approach):
    - Such a query consists of a sequence of words that make up a phrase
    - The documents returned must contain at least one instance of this phrase
  - Proximity queries (skipped-gram approach)
    - Is a relaxed version of the phase query and can be a combination of terms and phrases
    - These queries seek the documents that contain the search term in close proximity of each other

Information Retrieval and Search Engines
User Queries

- User Queries types:
  - Full document queries
    - Users search for documents that are similar to the query document
  - Natural language queries
    - This is the most complex case
    - The user asks a question and the IR system returns an answer to the that question
    - Used in question-answering systems (e.g., chatbots)

Information Retrieval and Search Engines
Architecture

- Query operation module
  - Can range between
    - Very Simple: just passes the query to the retrieval system
    - Complex: does preprocessing
- Indexer module
  - Used to index the original raw documents in some data structures
  - The data structures enables efficient retrieval
  - Most common is the inverted index

Information Retrieval and Search Engines
Architecture

- Retrieval system module:
  - Computes a relevance score (for the user query) for each retrieved documents
  - According the relevance the documents are ranked and presented to the user
- Document collection module:
  - A File System (FS), e.g., OSFS, HDFS, etc.
  - A database (Relational or NoSQL)

- The way in which terms and documents are represented governs the IR model
- There are three basic document representations:
  - Boolean model
  - Vector Space model
  - Statistical Language model
- Advance document representation:
  - Document Embeddings, e.g., Doc2Vec

- A document is represented using the bag of words (terms) model
- Given a set (collection) of n documents:
$$D = \{d_1, d_2, \dots, d_n\}, n = |D|$$
- All the distinct terms in the collection of documents can be modeled as a vocabulary:
$$V = \{t_1, t_2, \dots, t_m\}, m = |V|$$

- The entire corpus of documents can be represented as a matrix (document-term matrix) where:
  - The lines represent the document in D
  - The columns represent the terms in V
- Each cell in the matrix is a weight (w_{ij}) associated for the number of occurrences of a term in the document.
- A document $d_i \in D$ is modeled as a vector $d_i = \{w_{i1}, w_{i2}, \dots, w_{jm}\}$ where each w_{ij} is the weight associated to the term $t_j \in V$

- The document-term matrix is:

$$\begin{array}{c} \\ d_1 \\ d_2 \\ \vdots \\ d_n \end{array} \begin{array}{c} \begin{array}{cccc} t_1 & t_2 & \cdots & t_m \end{array} \\ \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1m} \\ w_{21} & w_{22} & \cdots & w_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nm} \end{pmatrix} \end{array}$$

- Boolean model
  - This is one of the simples models used to represent the weights in the document-term matrix
  - This model only considers if a term is present or not in a document, $w_{ij} \in \{0, 1\}$:

$$w_{ij} = \begin{cases} 1 & if\ t_j\ appears\ in\ d_i \\ 0 & otherwise \end{cases}$$

  - The document-term matrix will only contain 1s and 0s

- Boolean model
  - This model is used for Boolean queries
  - For example, given 3 terms $x, y\ and\ z$:
    - $(x\ AND\ y)\ AND\ (NOT\ z)$ says that a document must contain both x and y but not the term z
    - $x\ OR\ z$ says that a document must contain at least one of the terms x or z

- Vector space model
  - One of the best known and widely used IR models
  - Each weight is computed as a variation of the number of occurrences of the term in the document or collection of documents

- Vector space model most widely used representation:
  - $f_{t,d}$ the frequency of the term t in the document d is computed by counting
  - $TF(t, d)$ term frequency, the normalization of $f_{t,d}$
  - $TFIDF(t, d, D)$ term frequency – inverted document frequency
  - $Okapi\ BM25$ (BM stand for Best Match)
    - Okapi is the name of the information system proposed by Robertson, Spärck Jones, et al.

- $TF(t,d)$ term frequency is a normalization of the $f_{t,d}$
- Logarithmic normalized $TF(t,d)$:

$$TF(t,d) = \log(1 + f_{t,d})$$

- This has a bias towards longer documents because in longer documents terms that are irrelevant can appear multiple times, thus, they have a higher $TF(t,d)$

- To removes the bias towards longer documents, the augmented $TF(t,d)$

$$TF(t,d) = K + (1-K) \cdot \frac{f_{t,d}}{\max_{t' \in d} f_{t',d}}$$

- For $K = 0.5$ (determined empirically) we obtained the double normalization form of the $TF(t,d)$:

$$TF(t,d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{t' \in d} f_{t',d}}$$

- Another way to removes the bias towards longer documents is normalize the frequency of the term with the length of the document

$$TF(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

- The $IDF(t,D)$ is a measure of how much information a term provides:
  - Weather a term is common or rare across all documents
  - n is the number of documents in D
  - n_t is the number of documents where term t appears:

$$IDF(t,D) = 1 + \log \frac{n}{n_t}$$

- $TFIDF(t,d,D)$ is used to determine the importance of a term for a document in a corpus of documents:

$$TFIDF(t,d,D) = TF(t,d) \cdot IDF(t,D)$$

- Okapi BM25 is another weighting schema used to compute the weights for the document-term matrix

$$OkapiBM25(t,d,D) = \frac{TFIDF(t,d,D) \cdot (k_1+1)}{TF(t,d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avg_{d' \in D}(|d'|)}\right)}$$

- Where:
  - The free parameters values are:
    - $k_1 \in [1.2, 2.0]$
    - $b = 0.75$
  - $|d|$ is the length of the document
  - $avg_{d' \in D}(|d'|)$ is the average document length

- Ranking functions
  - Given a search query $Q = (q_1, q_2, \dots, q_k)$ where q_i are the query terms ($i = \overline{1,k}$, $k = |Q|$),
  - The objective is to retrieve the documents that contain the terms in the search query Q
  - We need to give a score that measures the importance of each document to the query
  - Then order the list of documents using this score and create a documents ranking

- The score for a document $d \in D$ is the sums of the weights ($TFIDF$, $OkapiBM25$) computed for each search term in query Q

$$TFIDF_{score}(Q,d,D) = \sum_{i=1}^{k} TFIDF(q_i,d,D)$$

$$OkapiBM25_{score}(Q,d,D) = \sum_{i=1}^{k} OkapiBM25(q_i,d,D)$$

- Using one of these scores, the documents' relevance to the query is determined

- Statistical Language Model
  - Are based on probability and have foundations in statistical theory
  - The input is
    - A set of n documents $D = \{d_1, d_2, \dots, d_n\}$
    - A query $Q = (q_1, q_2, \dots, q_k)$
  - We consider a query Q as being "generated" by a probabilistic model based on a document d_i
  - Using the Bayes rule, the rank of documents are estimated by the posterior probability $p(d_i|q)$:

$$p(d_i|Q) = \frac{p(Q|d_i) \cdot p(d_i)}{p(Q)}$$

- Statistical Language Model
  - For ranking, $p(Q)$ is not needed, as is the same for all the documents
  - The model assumes that each term is independently generated, which is essentially a multinomial distribution over words, and so:

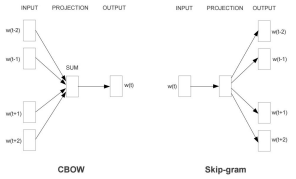
$$p(Q = (q_1, q_2, \dots, q_k)|d_i) = \prod_{j=1}^{k} p(q_j|d_i) = \prod_{j=1}^{m} p(t_j|d_i)^{\delta}$$

  - Where $\delta = \begin{cases} 1 & if\ q_j\ is\ present\ in\ d_i \\ 0 & otherwise \end{cases}$
  - So, the retrieve problem is reduced to estimating $p(t_j|d_i)$ $(t_j = q \in Q)$:

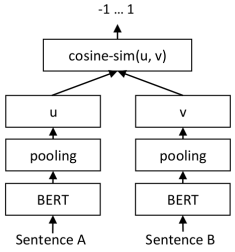
$$p(t_j|d_i) = \frac{f_{ij}}{|d_i|}$$

  - Where
    - f_{ij} is actually the frequency of word t_j in document d_i, f_{t_j,d_i}
    - $|d_i|$ is the length of document d_i

- Word representation models:
  - Word Embeddings:
    - Word2Vec Skip-Gram (SG)
    - Word2Vec Continuous Bag-of-Words (CBOW)
    - FastText SG
    - FastText CBOW
    - GloVe
  - Transformer Embeddings
    - BERT, RoBERTa, BART, etc.



- Document representation models:
  - Sentence/Document Embeddings
  - Sentence Transformers



- Before the documents in a collection are used for retrieval some preprocessing steps are usually performed
- These steps may include:
  - Expanding contractions
  - Sentence tokenization
  - Extracting terms
  - Removing stop words and punctuation

- Expanding contractions, i.e., shortened versions of the written and spoken forms of a word, syllable, or word group, created by omission of internal letters and sounds
- Sentence tokenization is the process by which the text of a document is split in sentences.
  - This step can be skipped if the terms are not processed further

- Extracting terms is the process by which each term is determined
  - Sometimes it is useful to extract the stem or the lemma of a word (not useful for opinion mining and sentiment analysis)
  - To extract the lemma sentence tokenization is used and also the part of speech tagger

- Extracting term – stemming
  - In many languages, a word has various forms depending on the context, e.g., verbs have the gerund form (-ing termination) or nouns can be singular or plural
  - These variations cause low retrieval because a relevant document can contain variation of a word but not the exact word
  - By removing the suffixes and prefixes of a word we can obtain the stem of the word
  - Stemming is the process of extracting the stems
  - The stems sometimes are not accurate, e.g., the words "cops" and "cope" are both reduce to "cop"

- Extracting term – lemmatization
  - A more accurate way of extracting the root word is by extracting the lemma, this process is called lemmatization
  - Lemmatization uses the part of speech of a word, the process that extracts the part of speech is called Part of Speech Tagging (PoS)
  - This is a very costly process, but it increases the accuracy of the retrieval

- Removing stop words and punctuation
  - Stop words are frequently occurring and insignificant words in a language, e.g.. the, a, an, etc.
  - These can be removes together with punctuation as they add no significate information to the process of information retrieval
  - For some tasks are important, e.g., opinion mining and sentiment analysis.

- The main purpose of IR is to retrieve documents given a search query
- One approach is to scan the collection of documents and to return the documents that match the query terms.
  - This method is actually impractical for a large collection of documents
- A second approach is to build a data structure (inverted index) that maps each word to the documents where it appears.

- How to construct an inverted index
  1. Given a collection of documents $D = \{d_1, d_2, \dots, d_n\}$, $n = |D|$
  2. Attach to each document an unique identifier $\{id_1, id_2, \dots, id_n\}$ where id_i is the unique identifier for document d_i
  3. Construct the vocabulary
  4. Attach to each word in the vocabulary the list of documents that contain the word

- In practice, an inverted index is a dictionary with the key the word and the value a list of documents.

$$IV_i = \{id_j | \, t_i \in d_i\}$$

- Sometimes, additional information can be stored in the inverted index, e.g.:
  - The term frequency of the word in the document
  - A list with positions, etc.

$$IV_i = \{< id_j, f_{ij}, [o_1, o_2, \dots, o_{|d_j|}] > | t_i \in d_i\}$$

- Where:
  - f_{ij} is the frequency of term t_i in document d_i
  - $o_k, k = \overline{1, |d_i|}$ is the position of term t_i in document d_i

- Example:

id_1: I am learning about inverted indexes.

id_2: Inverted indexes are used in information retrieval.

id_3: Applications that retrieve documents use inverted indexes.

- The vocabulary is:
  $\{learning, inverted, indexes, information, retrieval, applications, retrieve, documents\}$
- Notes:
  - the stop words were removed: I, am, about, are, used, in, that, use
  - Stemming or lemmatization can be applied, in this case it wasn't.

- Simple Inverted Index

$$learning: [id_1],$$
$$inverted: [id_1, id_2, id_3],$$
$$indexes: [id_1, id_2, id_3],$$
$$information: [id_2],$$
$$retrieval: [id_2],$$
$$applications: [id_3],$$
$$retrieve: [id_3],$$
$$documents: [id_3]$$

- Complex Inverted Index:

$$learning: [< id_1, 1, [3] >],$$
$$inverted: [< id_1, 1, [5] >, < id_2, 1, [1] >, < id_3, 1, [6] >],$$
$$indexes: [< id_1, 1, [5] >, < id_2, 1, [2] >, < id_3, 1, [6] >],$$
$$information: [< id_2, 1, [5] >],$$
$$retrieval: [< id_2, 1, [7] >],$$
$$applications: [< id_3, 1, [1] >],$$
$$retrieve: [< id_3, 1, [3]],$$
$$documents: [< id_3, 1, [4] >]$$

Information Retrieval and Search Engines
Inverted Index

- Searching with inverted indexes:
  - Given a search terms query each term is searched in the index and a concatenated list of all the document unique identifiers without duplicates is returned
  - Example:
    - For the search query 'applications information documents' the following documents are going to be returned:$\{id_3, id_2\}$
    - If a ranking function is used, then document d_3 will have a bigger weight that document d_2

Apache Lucene

- Apache Lucene™ is an open-source search software project
- The project releases are
  - Lucene™ core - the core search library written in Java
  - PyLucene - a python binding for Lucene.
- Lucene Core is a Java library providing
  - Powerful indexing
  - Search features
  - Spellchecking
  - Highlighting
  - Advanced analysis/tokenization capabilities
- PyLucene provides Python bindings for Lucene Core.

Apache Lucene

- Applications that can be developed using Apache Lucene:
  - Document analytics: Loading and traversing text documents based on some given criteria, finding the top terms from documents, aggregating on specific fields.
  - Log analytics: Analyzing application logs with high-performance dashboards built directly on top of Lucene.
  - Geospatial search: With the recent advent of geospatial queries and data structures, Lucene is fast becoming a popular choice for indexing latitudinal and longitudinal data and queries such as colocation searches.
  - Vector Database – improvements added by Elastic.co to make Elasticsearch a retrieval platform for Retrieval Augmented Systems (RAG)
- Two major search-based application have spawn from Lucene:
  - Apache Solr
  - Elasticsearch

Apache Lucene
Features

- Text/Language Analysis
  - The analysis is used to create appropriate internal representations for
    - Document to be indexed
    - Queries for searching documents
  - During indexing, tokens are created and stored in Lucene's Inverted Index
  - At query time, tokens are created to form appropriate query representations

Apache Lucene
Features

- Text/Language Analysis
  - The analysis consists in the following 3 steps:
    - Optional character filtering and normalization (e.g., removing diacritics)
    - Tokenization
    - Token filtering (e.g., stemming, lemmatization, stopword removal, n-gram creation)
  - This feature is used by Lucene for modeling documents

Apache Lucene
Features

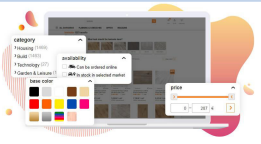
- Indexing and Storage
  - Used to index and store user defined documents
  - High degree of concurrency given by the lock-free indexing
  - Near real time indexing: documents become searchable as soon as they are indexed
  - Transactional support for additions and rollbacks
  - Different scoring models for terms, documents, and data sets

Apache Lucene
Features

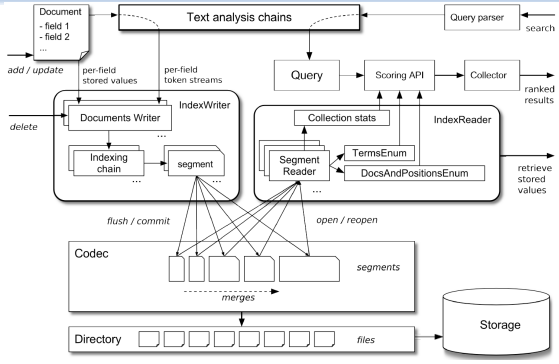
- Querying
  - Query options include the ability to filter, paginate, sort results, and perform relevance feedback
  - Multiple query representations, query parsers, and a query parsing framework for developer defined query parsers
  - Pluggable scoring model system for developers to implement their own
  - Predefined scoring models: traditional vector-space scoring model, Okapi BM25, Language Modeling, etc.

Apache Lucene
Features

- Developer libraries
  - Used for building search-based applications
  - Example libraries:
    - Result highlighting (snippet generation)
    - Faceting (show part of the analysis result) and Faceted Search (technique that involves augmenting traditional search techniques with a faceted navigation system)
    - Spatial search
    - Document grouping by key (e.g., group all documents with the same base URL together)
    - Document routing (via an optimized, in-memory, single document index)
    - Point-based spatial search
    - Auto-suggest



Apache Lucene Architecture



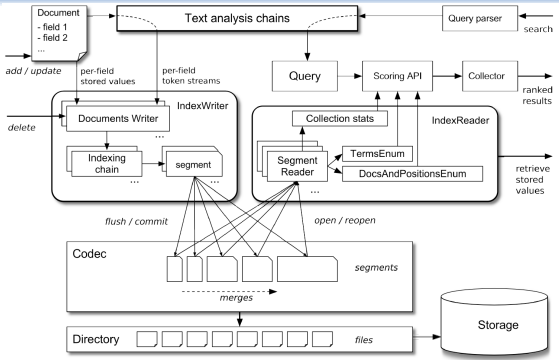
Apache Lucene Architecture

- Text Analysis
  - The text analysis chain produces a stream of tokens from the input data
  - Tokens are represented as a collection of attributes:
    - Term
    - Token position
    - Starting and ending offsets
    - Token type
    - Arbitrary payload data (a byte array to be stored in the index at the current position)
    - Integer flags
    - Other custom application-defined attributes (e.g., part-of-speech tags)

Apache Lucene Architecture

- Text Analysis
  - The text analysis chains consist of
    - Character filters (e.g., diacritics striping)
    - Tokenizers (which are the sources of token streams)
    - Series of token filters that modify the original token stream
  - Token streams perform specific functions such:
    - Tokenization by patterns, rules, and dictionaries (e.g., whitespace, regex, Chinese / Japanese / Korean)
    - Specialized token filters for efficient indexing of numeric values and dates (to support numerical range searching)
    - Language-specific stemming and stop word removal
    - Creation of character or word-level n-grams
    - Tagging, etc.

Apache Lucene Architecture

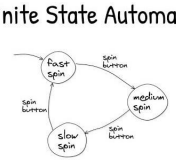


Apache Lucene Indexing Process

- Indexing
  - Lucene uses the well-known inverted index representation, with additional functionality for keeping adjacent non-inverted data on a per-document basis.
  - Both in-use and persistent data uses a variety of encoding schemas that affect the size of the indexed data and the cost of the data compression
  - Lucene uses pluggable mechanisms for data encoding (i.e., Codec API) and for the actual storage of index data (i.e., Directory API).
  - Incremental updates are supported and stored in index extents (i.e., segments) that are periodically merged into larger segments to minimize the total number of index parts

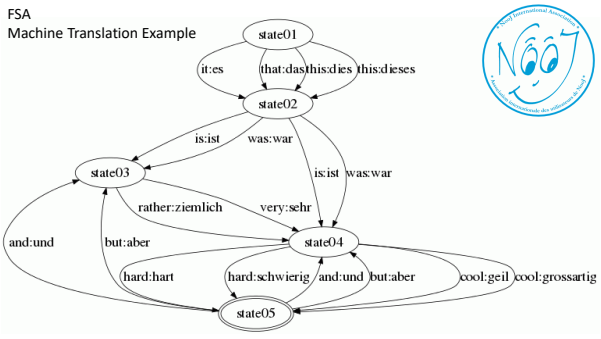
Apache Lucene Indexing Process

- Finite State Automata
  - The in-memory portion of the inverted index is implemented using a Finite State Transducer (FST)
  - An FST is a type of Finite State Automaton (FSA) that maps between two sets of symbols using two memory tapes:
    - Read a set of strings on the input tape
    - Generates a set of relations on the output tape



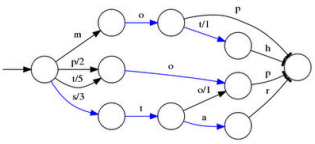
Apache Lucene Indexing Process

FSA
Machine Translation Example



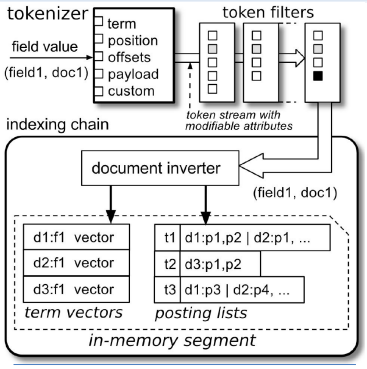
Apache Lucene Architecture

- The Finite State Transducer (FST) supports
  - Linear time construction of the minimal automaton
  - Compression
  - Reverse lookups
  - Weighted automata (used in NLP to assign weights to words and sentences)
  - Pluggable output algebras
  - Synonym processing
  - Spell correction
  - Auto-suggest/complete
  - Machine translation

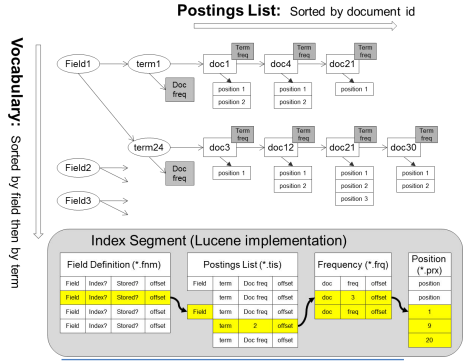


- Document Model
  - Documents
    - Are modeled in Lucene as a flat ordered list of fields with content
    - Are not required to have a unique identifier (though they often carry a field with this role for application-level unique key lookup)
    - During the indexing process, documents are assigned internal integer identifiers
  - Fields have a name, content data, a float weight (used later for scoring), and other attributes, depending on their type, which together determine how the content is processed and represented in the index
  - There can be multiple fields with the same name in a document, in which case they will be processed sequentially

- Field Types
  - There are two broad categories of fields in Lucene documents
  - Indexed fields contain content to be inverted
  - Stored fields contain content to be stored
- Fields may belong to either or both categories (e.g., with content both to be stored and inverted).
- Both fields can be submitted for storing / indexing
  - Only stored fields can be retrieved
  - Indexed/Inverted data can be accessed and traversed

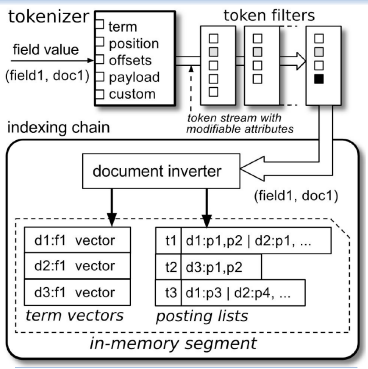


- Indexed fields can be
  - Provided in plain text, in which case it will be first passed through the text analysis pipeline to obtain token streams
  - The final form of a sequence of tokens with attributes (i.e., token streams)
- Token streams
  - Are then inverted and added to in-memory segments, which are periodically merged and flushed to disk
  - Depending on the field options, various token attributes (such as positions, starting / ending offsets and per-position payloads) are also stored with the inverted data.
  - It is possible to omit positional information while still storing the in-document term frequencies on a per-field basis
  - Use the Finite State Transducer for their in-memory representation



- A variant of an indexed field is a field where the creation and storage of term frequency vectors is requested
  - In this case the token stream is used also for building a small inverted index consisting of data from the current field only
  - This inverted data is then stored on a per-document and per-field basis
  - Term frequency vectors are particularly useful when performing document highlighting, relevance feedback or when generating search result snippets (region of text that best matches the query terms)

- Stored fields are typically used for storing auxiliary per-document data that is not searchable but would be cumbersome to obtain otherwise (e.g., it would require retrieval from a separate system)
- The per-document data is stored as byte arrays
  - Can be manipulated to be encoded using UTF-8 strings, numbers, arrays etc.
  - Optionally, it can be stored using strongly typed API (so called "doc values") that can use a more optimized storage format, i.e., strongly typed storage
  - This kind of strongly typed storage is used for example to store per-document and per-field weights (so called "norms", as they typically correspond to field length normalization factor that affects scoring)



Apache Lucene Indexing Process

- Indexing Chain
  - The resulting token stream
    - Is finally processed by the indexing chain
    - The supported attributes (term value, position, offsets and payload data) are added to the respective posting lists for each term
    - Posting list - a set of document unique indentifiers (ids) and offsets associated with each term in an inverted index
  - Term values
    - Can be arbitrary byte array values as terms
    - Can use custom comparators to define the sorting order of such terms
  - At this stage documents are assigned their internal document identifiers, i.e., small sequential integers (for efficient delta compression)
  - These identifiers
    - Are used for identifying document data within a particular segment
    - Change after two or more segments are merged (during index compaction)

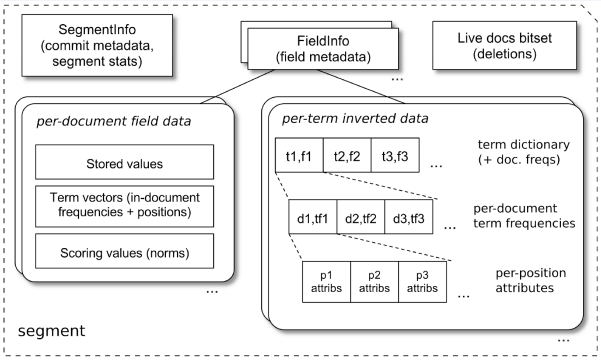
Apache Lucene Indexing Process

- Incremental Index Updates
  - Indexes can be updated incrementally on-line
    - Simultaneously with searching
    - By adding new documents and/or deleting existing ones (sub-document updates are a work in progress)
  - Index extents
    - Are a common way to implement incremental index updates
    - Do not require modifying the existing parts of the index

Apache Lucene Indexing Process

- When new documents are submitted for indexing
  - It passes through the indexing chain
  - The resulting inverted and non-inverted data is accumulated in new in-memory index extents called segments
  - The segment use a compact in-memory representation (a variant of Codec)
  - Periodically, these in-memory segments are flushed to a persistent storage (using the Codec API and Directory API abstractions)
  - The flushing appears whenever a configurable threshold is reached, e.g., the total number of documents, the size in bytes of the segment, etc.

Apache Lucene Segment



Apache Lucene Indexing Process

- The Indexer Writer
  - The Index Writer is a high-level (Java) class responsible for
    - Processing index updates (additions and deletions)
    - Recording them in new segments and creating new commit points
    - Triggering the index compaction (segment merging)
  - It uses a pool of DocumentWriter-s that create new in-memory segments
  - As new documents are being added and in-memory segments are being flushed to storage, periodically an index compaction (merging) is executed in the background that reduces the total number of segments that comprise the whole index

Apache Lucene Indexing Process

- The Indexer Writer
  - Document deletions
    - Are Queries that select the documents to be deleted
    - Are accumulated:
      - Applied to the in-memory segments before flushing (while they are still mutable)
      - Recorded in a commit point so that they can be resolved when reading the already flushed immutable segments

Apache Lucene Indexing Process

- The Indexer Writer
  - The commit point:
    - Is created at each flush operation or index compaction
    - Is recorded in a global index structure using a two-phase commit
    - Is a list of segments and deletions comprising the whole index at the point in time when the commit operation was successfully completed
  - At the commit point, the segment data that is being flushed from in-memory segments is encoded using the configured Codec implementation

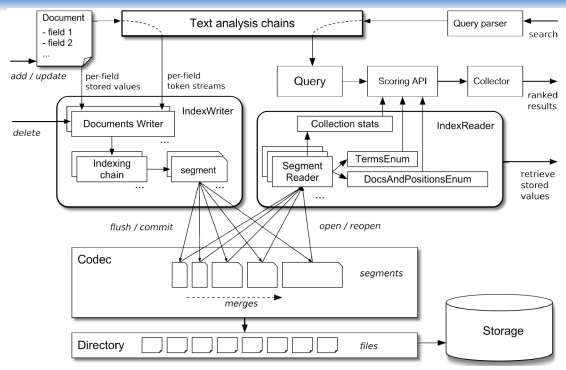
Apache Lucene Indexing Process

- The Indexer Writer
  - The segment data
    - Are fully immutable (write-once)
    - Any changes are expressed either as
      - New segments
      - New lists of deletions
    - Both types of changes create new commit points
  - The updated view of the latest version of the index becomes visible when a commit point is recorded using a two-phase commit
    - This enables lock-free reading operations concurrently with updates
    - Point-in-time travel by opening the index for reading using some existing past commit point

Apache Lucene Indexing Process

- As segments are immutable
  - Deletions do not actually remove data from existing segments
  - Instead, the delete operations are resolved when existing segments are open
  - Thus, the deletions are represented as a bitset of live (not deleted) documents
  - This bitset is then used when enumerating posting lists and stored fields and during search to hide deleted documents.
  - Global index statistics are not recalculated, so they are slightly wrong (still include term statistics from deleted documents)
  - For performance reasons the data of deleted documents is actually removed only during segment merging and the global index statistics are recalculated

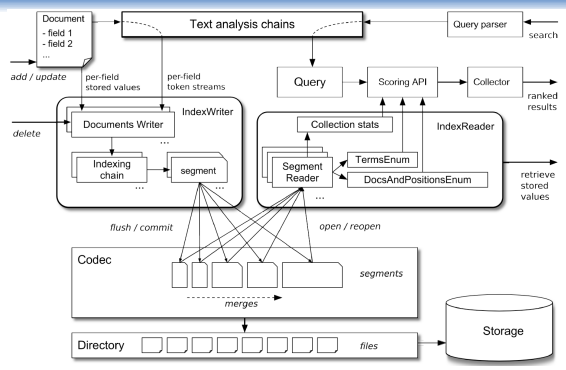
Apache Lucene Querying



Apache Lucene Querying

- The Indexer Reader
  - Provides high-level methods to retrieve stored fields, term vectors, and to traverse the inverted index data
  - Uses the Codec API to retrieve and decode the index data
  - Represents the view of an index at a specific point in time
- A user obtains information using an Index Reader from
  - A commit point when all data has been written to disk
  - Directly from Index Writer: a near real-time snapshot that includes both the flushed and the in-memory segments

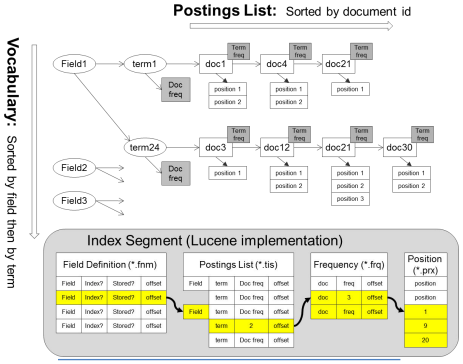
Apache Lucene Codec API



Apache Lucene Codec API

- The Codec API deals with
  - Index data encoding for writing in the storage
  - Index data decoding for reading
  - Segment data merging
- A 4-D View of the Inverted Index
  - The Codec API encapsulates the inverted index data as a logical 4-dimensional table that can be traversed using enumerators
  - The dimensions are: field, term, document, and position
  - Using a cursor, we can advance along rows and columns of this table in each dimension

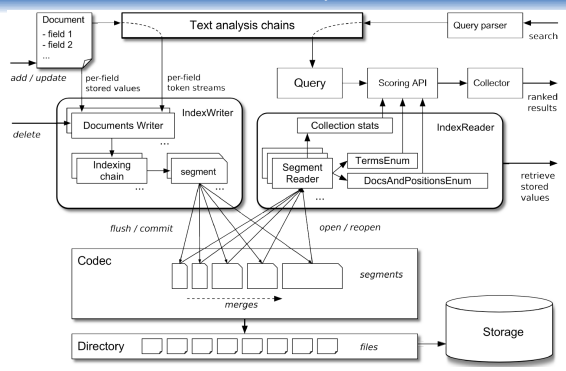
Apache Lucene Indexing Process



Apache Lucene Codec API

- Lucene Codecs API
  - The default codec implementation uses a combination of well-known compression algorithms and strategies selected to provide a good tradeoff between index size (and related costs of I/O seeks) and coding costs
  - The codes offer good balance between high performance indexing and fast execution of queries.

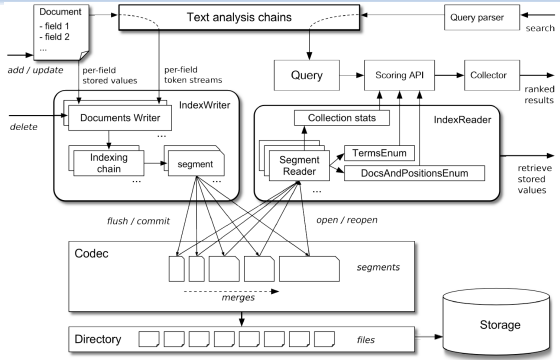
Apache Lucene Directory



Apache Lucene
Directory API

- Directory API
  - The physical I/O access is abstracted using the Directory API
  - The Directory API offers a very simple file system-like views of the persistent storage
  - The Lucene Directory is basically a flat list of files
  - Files are write-once
  - Abstractions are provided for sequential and random access for writing and reading of files

Apache Lucene
Searching



Apache Lucene
Searching

- Searching can be broken down as follows
  - Query model
  - Query types
  - Query evaluation
  - Scoring
  - Common search extensions

Apache Lucene
Query Model

- Query model
  - Lucene does not enforce a particular query language
  - Query objects are used to perform searches
  - Several Queries are provided as building blocks to express complex queries
  - Developers can construct their own query language programmatically or via a Query Parser.

Apache Lucene
Query Types

- Query types used in Lucene:
  - Term queries that evaluate a single term in a specific field
  - Boolean queries
  - Proximity queries
  - Position-based queries that allow to express more complex rules for proximity and relative positions of terms
  - Wildcard, fuzzy and regular expression queries that use automata for evaluating matching terms
  - Disjunction-max query that assigns scores based on the best match for a document across several fields
  - Payload query that processes per-position payload data, etc.

Apache Lucene
Query Functions & Search Queries

- Function queries
  - Lucene also supports the incorporation of field values into scoring, i.e., function queries
  - Function queries can be used to add useful scoring factors like time and distance into the scoring model
- Search query are either
  - Parsed by a Query Parser into a Query tree
  - Generated and combined programmatically

Apache Lucene
Query Evaluation

- Query evaluation
  - At Query execution time, the Scoring API
    - Processed each inverted index segment sequentially for efficiency
    - Calculates a score for each document
  - Documents are scored using one of two strategies:
    - Document-at-a-Time (DAAT)
    - TAAT (Term-at-a-Time) only when the number of terms is low.
  - A Collector is responsible for actually consuming these Scorers and doing something with the results, for example populating a priority queue of the top-N documents

Apache Lucene
Scoring

- Scoring is done using a Similarity class that implements a policy for scoring terms and queries
- When computing a score, the Similarity class considers
  - Term and global index statistics
  - Query specifics (e.g., distance between terms of a phrase, Levenshtein edit distance of fuzzy terms, etc.)
- Lucene scoring uses well-known scoring models: TFIDF, BM25, etc.



Apache Lucene
Common Search Extensions

- Common Search Extensions
  - Lucene provides extended query processing capabilities to support easier navigation of search results:
    - Rollup/drilldown capabilities using the Faceting module
    - Result grouping supports folding related documents into a single combined result.
    - Support for nested documents, query expansion, and geospatial search